

UNIT – 2

OOPS USING JAVA

1. Constants, Variables and Data Types

- 1.1 **Constants** - A **constant** is a value that does not change during program execution.

Example –

```
class ConstantExample {  
    public static void main(String[] args) {  
  
        final double PI = 3.14;  
  
        System.out.println("Value of PI: " + PI);  
    }  
}
```

`final` keyword makes the value constant.

- 1.2 **Variables** – A variable is a named memory location used to **store data that can change during program execution**.

Syntax - `dataType variableName;`

Example - `int age; double salary;`

- **Declaration of Variables** - Declaring a variable means **specifying the variable name and its data type**.

Syntax: `dataType variableName;`

- **Giving Values to Variables (Initialization)** - Assigning a value to a variable is called initialization.

Syntax : `variableName = value;`

Example:

```
int age;  
age = 20;
```

- **Declaration + Initialization**

```
int age = 20;
```

- 1.3 **Data Types** : A data type **specifies the type of data** a variable can store.

Example:

- Integer, decimal, character

Types of Data Types in Java

1. Primitive Data Types

These are **basic built-in data types**.

Data Type	Size	Example
Int	4 bytes	10
Float	4 bytes	10.5
Double	8 bytes	12.45
Char	2 bytes	'A'
Boolean	1 bit	true/false

Example

```
class PrimitiveExample {  
    public static void main(String[] args) {  
  
        int age = 20;  
        float price = 10.5f;  
        char grade = 'A';  
        boolean status = true;  
  
        System.out.println(age);  
        System.out.println(price);  
        System.out.println(grade);  
        System.out.println(status);  
    }  
}
```

2. Non-Primitive Data Types

These are **derived data types created by programmers**.

Examples:

- String
- Array

- Class
 - Object
-

Type Casting: Type casting means converting one data type into another data type.

Types of Type Casting

1. Implicit Casting (Widening)

Automatic conversion from **smaller type to larger type**.

Example

```
class TypeCastingExample {  
    public static void main(String[] args) {  
  
        int a = 10;  
        double b = a; // automatic conversion  
  
        System.out.println(b);  
    }  
}
```

2. Explicit Casting (Narrowing)

Manual conversion from **larger type to smaller type**.

Syntax

dataType variable = (dataType)value;

Example

```
class TypeCastingExample2 {  
    public static void main(String[] args) {  
  
        double a = 10.5;  
        int b = (int)a;  
  
        System.out.println(b);  
    }  
}
```

2. Operators and Expressions

2.1 Operators are symbols used to perform operations on variables and values.

Types of Operators

1. Arithmetic Operators: Arithmetic operators are used to perform mathematical calculations such as addition, subtraction, multiplication and division.

They operate on numeric values and return a numeric result.

Operators: + - * / %

Example

```
class ArithmeticExample {
    public static void main(String[] args) {

        int a = 10;
        int b = 5;

        System.out.println("Addition: " + (a+b));
        System.out.println("Subtraction: " + (a-b));
        System.out.println("Multiplication: " + (a*b));
        System.out.println("Division: " + (a/b));
    }
}
```

Output

Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3
Modulus: 1

2. Relational Operators

Relational operators are used to **compare two values or variables**.

The result of relational operations is always **true or false (boolean value)**.

Operators: > < >= <= == !=

Example -

```
class ArithmeticExample {
    public static void main(String[] args) {

        int a = 10;
        int b = 20;

        System.out.println(a>b);
        System.out.println(a<b);
    }
}
```

```
System.out.println( a>=b);
System.out.println(a<=b);
System.out.println( a==b);
System.out.println(a!=b);}}
```

Output

```
false
true
false
true
false
true
```

3. Logical Operators

Logical operators are used to **combine two or more conditions** in a program. They return **true or false depending on logical evaluation**.

Operators: && || !

Example

```
class LogicalOperators
{
    public static void main(String args[])
    {
        int a = 10;
        int b = 20;

        System.out.println((a < b) && (a > 5));
        System.out.println((a > b) || (a < b));
        System.out.println(!(a < b));
    }
}
```

Output

```
true
true
false
```

4. Assignment Operators

Assignment operators are used to **assign values to variables**. They can also **combine arithmetic operations with assignment**.

Operators: = += -= *= /= %=

Example

```
class AssignmentOperators
{
public static void main(String args[])
{
    int a = 10;

    a += 5;
    System.out.println("+= : " + a);

    a -= 3;
    System.out.println("-= : " + a);

    a *= 2;
    System.out.println("*= : " + a);

    a /= 4;
    System.out.println("/= : " + a);

    a %= 3;
    System.out.println("%= : " + a);
}
}
```

Output

```
+= : 15
-= : 12
*= : 24
/= : 6
%= : 0
```

5. Increment and Decrement Operators

Increment and decrement operators are used to **increase or decrease the value of a variable by 1**. They are commonly used in **loops and counters**.

Operators: ++ --

Example

```
class IncrementDecrement
{
public static void main(String args[])
{
    int a = 5;
```

```

        a++;
        System.out.println("Increment: " + a);

        a--;
        System.out.println("Decrement: " + a);
    }
}

```

Output

Increment: 6
Decrement: 5

6. Conditional (Ternary) Operator

The conditional operator is a **short form of if-else statement**.
It evaluates a condition and **returns one of two values**.

Operator: ?:

Example

```

class ConditionalOperator
{
    public static void main(String args[])
    {
        int a = 10;
        int b = 20;

        int max = (a > b) ? a : b;

        System.out.println("Maximum: " + max);
    }
}

```

Output

Maximum: 20

7. Bitwise Operators

Bitwise operators perform operations on **binary bits of integer numbers**.
They are mainly used in **low-level programming and bit manipulation**.

Operators: & | ^ ~ << >>

Example

```
class BitwiseOperators
{
    public static void main(String args[])
    {
        int a = 5;
        int b = 3;

        System.out.println("AND: " + (a & b));
        System.out.println("OR: " + (a | b));
        System.out.println("XOR: " + (a ^ b));
        System.out.println("Left Shift: " + (a << 1));
        System.out.println("Right Shift: " + (a >> 1));
    }
}
```

Output

AND: 1
OR: 7
XOR: 6
Left Shift: 10
Right Shift: 2

2.2 Expressions

An **expression** is a combination of **variables, operators and values**.

Example

$a + b * c$

Arithmetic Expression Example

```
int result = a + b * c;
```

- **Evaluation of Expressions**

Java evaluates expressions according to **operator precedence rules**.

Example

$10 + 5 * 2$

First multiplication → then addition.

Result

- **Operator Precedence**

Order of execution of operators.

Example priority:

1. ()
 2. *, /
 3. +, -
 4. =
 5. relational operators
-

- **Associativity**

Associativity determines **direction of evaluation** when operators have same precedence.

Example

$a + b + c$

Evaluated from **left to right**.

3. Decision Making, Branching and Looping

3.1 Decision Making

Decision making allows a program to **choose different paths based on conditions**.

1. If Statement

The if statement is used to execute a block of code only when a condition is true.
If the condition is false, the statements inside the if block are skipped.

Example

```
class IfExample
{
    public static void main(String args[])
```

```
    {  
        int age = 20;  
  
        if(age >= 18)  
        {  
            System.out.println("Eligible to vote");  
        }  
    }  
}
```

Output

Eligible to vote

2. If-Else Statement

The if-else statement is used when **two alternative actions are possible**.
If the condition is true, the if block executes otherwise the else block executes.

Example

```
class IfElseExample  
{  
    public static void main(String args[])  
    {  
        int num = 7;  
  
        if(num % 2 == 0)  
            System.out.println("Even");  
        else  
            System.out.println("Odd");  
    }  
}
```

Output

Odd

3. Switch Statement

The switch statement is used to select one block of code from multiple choices.
It compares the variable with different case values.

Example

```
class SwitchExample  
{
```

```

public static void main(String args[])
{
    int day = 2;

    switch(day)
    {
        case 1:
            System.out.println("Monday");
            break;

        case 2:
            System.out.println("Tuesday");
            break;

        case 3:
            System.out.println("Wednesday");
            break;

        default:
            System.out.println("Invalid");
    }
}

```

Output

Tuesday

3.2 Looping Statements: Loops are used to **execute a block of code repeatedly**.

1. For Loop

The for loop is used when the number of iterations is known beforehand. It contains initialization, condition checking and increment/decrement in one statement.

Syntax

```

for(initialization; condition; increment)
{ statement;
}

```

Example

```

class ForLoop
{
    public static void main(String args[])
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println(i);
        }
    }
}

```

Output

1
2
3
4
5

2. While Loop

The while loop executes a block of code **as long as the condition is true**. It is mostly used when the number of iterations is **not known in advance**.

Example-

```
class WhileLoop
{
    public static void main(String args[])
    {
        int i = 1;

        while(i<=5)
        {
            System.out.println(i);
            i++;
        }
    }
}
```

Output

1
2
3
4
5

3. Do While Loop

The do-while loop executes the loop **at least once before checking the condition**. After execution, the condition is checked for further iterations.

Program

```
class DoWhileLoop
{
    public static void main(String args[])
    {
        int i = 1;
```

```
        do
        {
System.out.println(i);
        i++;
        }
        while(i<=5);
        }
    }
```

Output

```
1
2
3
4
5
```

Jump Statements in Loops

1.Break: The break statement is used to terminate the loop immediately. Control moves to the statement after the loop.

Example -

```
class BreakExample
{
    public static void main(String args[])
    {
        for(int i=1;i<=5;i++)
        {
            if(i==3)
                break;

            System.out.println(i);
        }
    }
}
```

Output

```
1
2
```

2.Continue: The continue statement is used to **skip the current iteration of a loop**.
The loop continues with the next iteration.

Program

```
class ContinueExample
{
public static void main(String args[])
{
    for(int i=1;i<=5;i++)
    {
        if(i==3)
            continue;

        System.out.println(i);
    }
}
```

Output

1
2
4
5
